

INSTITUTO CATÓLICO KAROL WOJTYLA

Actividad desarrollada:

CONSULTAS, VALIDACIÓN Y CONSOLIDACIÓN DE LA BASE DE DATOS

Grado:

Tercer Año de Bachillerato Técnico en Desarrollo de Software

MÓDULO 3.3: Administración de Bases de Datos

Alumno:

José Rolando Velasco Peña

Docente:

Lic. Criseyda Guadalupe Araujo Mengar

Cabañas Oeste, El Salvador

25 de septiembre de 2026

ÍNDICE

ÍNDICE	2
TABLA DE FIGURAS	3
1. INTRODUCCIÓN	4
2. DATOS DEL ESTUDIANTE Y ROL	5
3. CONSULTAS INNER JOIN.....	6
3.1. Consulta 1 INNER JOIN:	6
3.1.1. Explicación:	6
3.1.2. Evidencia de ejecución:	6
3.2.1. Explicación:	7
3.2.2. Evidencia de ejecución:	8
4. CONSULTAS DE RESUMEN	9
5. VALIDACIÓN DE LOS DATOS	14
6. CONSOLIDACIÓN DEL TRABAJO DEL EQUIPO	19
7. ACTUALIZACIÓN EN GITHUB	20
7.1. Branch (rama personal utilizada)	20
7.2. Commits	21
7.3. Archivos modificados	21
7.3.1. Archivo: 03_consultas_alertas	22
7.3.1. Archivo: 03_consultas_alertas	22
8. REGISTRO DE MI APORTE	23
9. CONCLUSIÓN	25

TABLA DE FIGURAS

Figura 1 Sintaxis SQL, para la consulta uno utilizando INNER JOIN.....	6
Figura 2 Captura del resultado de la ejecución de la consulta 1 con INNER JOIN	7
Figura 3 Sintaxis SQL, para la consulta dos utilizando INNER JOIN.....	8
Figura 4 Captura del resultado de la ejecución de la consulta dos con INNER JOIN	8
Figura 5 Consulta con COUNT para obtener la cantidad de emergencias activas.	9
Figura 6 Resultado obtenido de la consulta de agregación utilizando COUNT.....	10
Figura 7 Consulta utilizado GROUP BY para agrupar las emergencias por tipo.....	11
Figura 8 Resultado obtenido de la consulta de agregación utilizando GROUP BY.	11
Figura 9 Consulta con GROUP BY para agrupar registros por estado e institución.....	12
Figura 10 Captura del resultado obtenido de la consulta con GROUP BY.	12
Figura 11 Consulta SQL con GROUP BY, junto con la condición HAVING	13
Figura 12 Resultado obtenido de la consulta realizada con GROUP BY y HAVING.	14
Figura 13 Inconsistencia 1 de estados contradictorios entre estado y comado.	15
Figura 14 Solución aplicada para inconsistencia 1.....	15
Figura 15 Captura de comprobación de datos actualizados correctamente	16
Figura 16 Inconsistencia 2 duplicidad de datos.	16
Figura 17 Corrección aplicada para la inconsistencia número 2.....	17
Figura 18 Comprobación de la solución aplicada a la inconsistencia 2.	17
Figura 19 Inconsistencia 3 falta de restricción de datos en columna "estado".	18
Figura 20 Corrección aplicada para la inconsistencia 3.....	19
Figura 21 Captura de la rama utilizada durante las actividades.	21
Figura 22 Evidencia de commits realizados en el repositorio.	21
Figura 23 Archivo modificado "02_datos_prueba_alertas".....	22
Figura 24 Archivo modificado "03_consultas_alertas".	22

1. INTRODUCCIÓN

El presente documento presenta las evidencias de las actividades prácticas desarrolladas durante, donde se realizó consultas avanzadas en la base de las emergencias que registrara el sistema SafeSchool Alert, utilizando INNER JOIN, LEFT JOIN, y funciones de agregación, así como métodos de agrupación para obtener datos por medio de agrupación y más completa de cada emergencia registrada en la institución educativa.

Además, se realizó una revisión de los datos almacenados en la tabla alertas, así como los datos, y estructura de la base de datos, para identificar inconsistencias en los datos almacenados en cada campo de una emergencia, para corregir los errores y registros que necesitaban correcciones por duplicidad, lo que generaba confusión y campos que almacenaban información contradictoria en el mismo registro. También se aplicaron modificaciones y restricciones para mejorar la integridad y consistencia de la información almacenada en la base de datos.

2. DATOS DEL ESTUDIANTE Y ROL

Con el propósito de trabajar de forma colorativa para el diseño y estructura de la base de datos que se utilizara para el proyecto SafeSchool Alert, como parte de los integrantes del equipo se me asignaron los cargos de gestionar y supervisar la tabla alerto, que almacena los registros de cada una de las emergencias reportadas y detectas dentro de la institución educativa. Para ello en esta sección se presenta los datos principales que se me asignaron trabajar en la base de datos del proyecto, durante esta actividad práctica de la semana número 3:

N°	Dato	Información
1	Proyecto	SafeSchool Alert
2	Estudiante	José Rolando Velasco Peña
3	Módulo	BTVDS 3.3 Administración de Bases de Datos
4	Semana	3 consultas <u>avanzadas</u>
5	Rol	Consultas, verificación de inconsistencias en la tabla " <u>alertas</u> "
6	Rama de trabajo	Rolando_SafeSchool_DB
7	Repositorio de la BD	https://github.com/joserolandovelascopena-code/SafeSchool-Alert-BD

3. CONSULTAS INNER JOIN

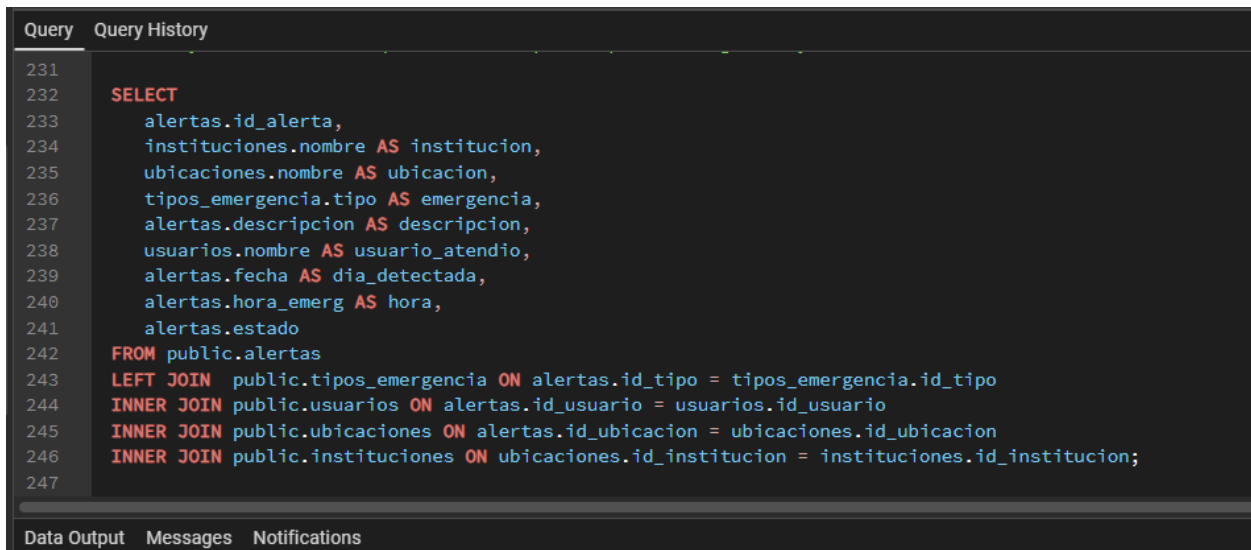
3.1. Consulta 1 INNER JOIN:

Mostrar información detallada de las emergencias, incluyendo el usuario que la atendió, el tipo de emergencia y su ubicación.

3.1.1. Explicación:

La primera consulta realizada para en la práctica constituye a una consulta avanzada utilizando LEFT JOIN para traer todos los registros existentes en la tabla alertas para que ninguna emergencia quede sin ser consultada. Para consultar los datos como: el estado, ubicación e institución a quien pertenece cada emergencia se hizo uso del INNER JOIN, el cual nos permite traer los registros externos en las tablas “tipos_emergencia, instituciones y ubicaciones”, que se relacionen con una emergencia en “alertas” mostrara esos datos al realizar dicha consulta:

3.1.2. Evidencia de ejecución:



```
231
232 SELECT
233     alertas.id_alerta,
234     instituciones.nombre AS institucion,
235     ubicaciones.nombre AS ubicacion,
236     tipos_emergencia.tipo AS emergencia,
237     alertas.descripcion AS descripcion,
238     usuarios.nombre AS usuario_atendio,
239     alertas.fecha AS dia_detectada,
240     alertas.hora_emerg AS hora,
241     alertas.estado
242 FROM public.alertas
243 LEFT JOIN public.tipos_emergencia ON alertas.id_tipo = tipos_emergencia.id_tipo
244 INNER JOIN public.usuarios ON alertas.id_usuario = usuarios.id_usuario
245 INNER JOIN public.ubicaciones ON alertas.id_ubicacion = ubicaciones.id_ubicacion
246 INNER JOIN public.instituciones ON ubicaciones.id_institucion = instituciones.id_institucion;
247
```

Figura 1 Sintaxis SQL, para la consulta uno utilizando INNER JOIN

Resultado: El resultado obtenido al ejecutar la instrucción SQL fue una tabla con los datos de cada emergencia más completa y profesional, mostrando la ubicación en que origino la emergencia, el tipo y el usuario que la atendió.

Query

Query History

232

SELECT

233

alertas.id_alerta,

234

instituciones.nombre,

235

ubicaciones.nombre AS ubicacion,

236

tipos_emergencia.tipo AS emergencia,

237

alertas.descripcion AS descripcion,

238

usuarios.nombre AS usuario_atendio,

239

alertas.fecha AS dia_detectada,

240

alertas.hora_emerg AS hora,

241

alertas.estado

242

FROM public.alertas

243

LEFT JOIN public.tipos_emergencia **ON** alertas.id_tipo = tipos_emergencia.id_tipo

244

INNER JOIN public.usuarios **ON** alertas.id_usuario = usuarios.id_usuario

245

INNER JOIN public.ubicaciones **ON** alertas.id_ubicacion = ubicaciones.id_ubicacion

246

INNER JOIN public.instituciones **ON** ubicaciones.id_institucion = instituciones.id_institucion;

Data Output

Messages

Notifications

Figura 2 Captura del resultado de la ejecución de la consulta 1 con INNER JOIN

3.2. Consulta 2 INNER JOIN:

Mostrar las emergencias registradas en el Edificio A, indicando el tipo de emergencia, su descripción, fecha, hora y estado, ordenadas de la más reciente a la más antigua.

3.2.1. Explicación:

La segunda consulta realizada en la práctica constituye en consultar únicamente las emergencias correspondientes al “Edificio A” y obtener los datos como: tipo de emergencia, ubicación, fecha, hora y estado, se hizo uso del INNER JOIN, mediante el cual fue posible relacionar los registros de las tablas “tipos_emergencia” y “ubicaciones” con los registros de la tabla “alertas” que coinciden entre ellas.

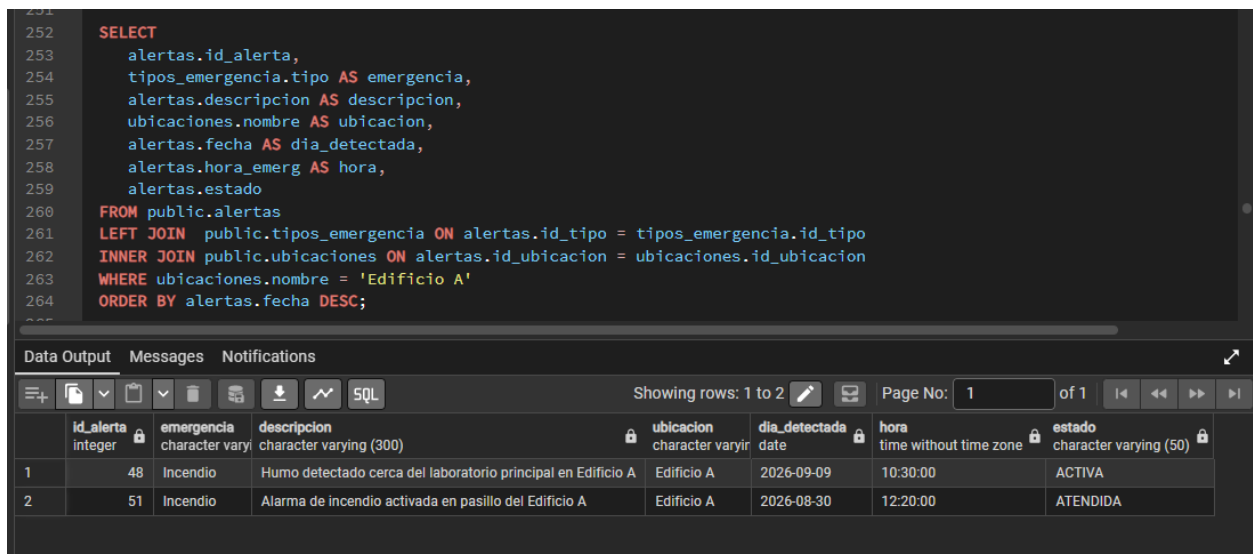
Se utilizó la condición WHERE para filtrar únicamente las emergencias que pertenecen al “Edificio A” y ORDER BY para ordenar los resultados desde la emergencia más reciente hasta la más antigua

3.2.2. Evidencia de ejecución:

```
1 SELECT
2     alertas.id_alerta,
3     tipos_emergencia.tipo AS emergencia,
4     alertas.descripcion AS descripcion,
5     ubicaciones.nombre AS ubicacion,
6     alertas.fecha AS dia_detectada,
7     alertas.hora_emerg AS hora,
8     alertas.estado
9 FROM public.alertas
10 LEFT JOIN public.tipos_emergencia ON alertas.id_tipo = tipos_emergencia.id_tipo
11 INNER JOIN public.ubicaciones ON alertas.id_ubicacion = ubicaciones.id_ubicacion
12 WHERE ubicaciones.nombre = 'Edificio A'
13 ORDER BY alertas.fecha DESC;
```

Figura 3 Sintaxis SQL, para la consulta dos utilizando INNER JOIN

Resultado: El resultado obtenido al ejecutar la instrucción SQL fue una tabla con las emergencias registradas en el “Edificio A”, mostrando información como el tipo de emergencia, descripción, ubicación, fecha, hora y estado, ordenadas desde la más reciente hasta la más antigua.



	id_alerta	emergencia	descripcion	ubicacion	dia_detectada	hora	estado
1	48	Incendio	Humo detectado cerca del laboratorio principal en Edificio A	Edificio A	2026-09-09	10:30:00	ACTIVA
2	51	Incendio	Alarma de incendio activada en pasillo del Edificio A	Edificio A	2026-08-30	12:20:00	ATENDIDA

Figura 4 Captura del resultado de la ejecución de la consulta dos con INNER JOIN

4. CONSULTAS DE RESUMEN

En la presente sección se presentan las consultas implementadas utilizando funciones de agregación como (COUNT) y la agrupación de filas con GROUP BY. Cabe recalcar que en las consultas presentadas no se utilizaron funciones como: AVG, SUM, porque la información almacenada en la tabla alertas, corresponde a tipos de datos (varchar), y estas funciones trabajan con tipos de datos numéricos, lo que hace que no se puedan aplicar directamente en la tabla alertas, por la razón de que no aceptan tipos de datos de texto.

4.1. Consulta 1: ¿Cuántas emergencias por tipo se encuentran activas?

4.1.2. Explicación: Para darle respuesta a esta pregunta relacionada con las necesidades reales que el sistema puede tener, se utilizó COUNT Y GROUP BY, para agrupar exactamente todos los registros de alertas, agrupadas por tipo de emergencia y para mostrar solo las emergencias activas se utilizó WHERE para mostrar solo emergencias de estado activa y contar cuántas existen por tipo:

```
SELECT
    alertas.estado,
    tipos_emergencia.tipo,
    COUNT(*) AS cant_emergencias_activas
FROM public.alertas
LEFT JOIN public.tipos_emergencia ON alertas.id_tipo = tipos_emergencia.id_tipo
WHERE alertas.estado = 'ACTIVA'
GROUP BY alertas.estado, tipos_emergencia.tipo;
```

Figura 5 Consulta con COUNT para obtener la cantidad de emergencias activas.

4.1.2. Resultado: Al realizar la ejecución de la consulta SQL, se obtuvieron exitosamente 2 registros agrupados por estado y tipo de emergencia, mostrando la cantidad exacta de emergencias registradas con estado “ACTIVA” según su tipo de emergencia (médica, seguridad e incendio).

```

270
271 SELECT
272     alertas.estado,
273     tipos_emergencia.tipo,
274     COUNT(*) AS cant_emergencias_activas
275 FROM public.alertas
276 LEFT JOIN public.tipos_emergencia ON alertas.id_tipo = tipos_emergencia.id_tipo
277 WHERE alertas.estado = 'ACTIVA'
278 GROUP BY alertas.estado, tipos_emergencia.tipo;
279

```

	estado character varying (50)	tipo character varying (100)	cant_emergencias_activas bigint
1	ACTIVA	Incendio	1
2	ACTIVA	Medica	1

Total rows: 2 Query complete 00:00:00.344 CRLF Ln 268, Col 1

Showing rows: 1 to 2 Page No: 1 of 1

✓ Successfully run. Total query runtime: 324 msec. 2 rows affected.

Figura 6 Resultado obtenido de la consulta de agregación utilizando COUNT

4.2. Consulta 2: ¿Cuántas emergencias han ocurrido en el la institución educativa según su tipo?

4.2.1. Explicación: En esta segunda consulta, se utilizó igualmente COUNT y GROUP BY, para contar y agrupar todos los registros de emergencias según el tipo de emergencia ocurrido dentro de la institución educativa.

Además, para consultar datos completos y consistentes se utilizó LEFT JOIN e INNER JOIN para relacionar los registros de la tabla “alertas” con las tablas “tipos_emergencia”, “ubicaciones” e “instituciones”, lo que permitió identificar a qué institución pertenece cada emergencia. Finalmente, se utilizó WHERE para mostrar únicamente las emergencias pertenecientes a la institución “Instituto Católico Karol Wojtyla”.

```

282
283 SELECT
284     i.nombre AS institucion,
285     te.tipo,
286     COUNT(*) AS cant_emergencias
287 FROM public.alertas a
288 LEFT JOIN public.tipos_emergencia te ON a.id_tipo = te.id_tipo
289 INNER JOIN public.ubicaciones u ON a.id_ubicacion = u.id_ubicacion
290 INNER JOIN public.instituciones i ON u.id_institucion = i.id_institucion
291 WHERE i.nombre = 'Instituto Católico Karol Wojtyla'
292 GROUP BY i.nombre, te.tipo;
293

```

Figura 7 Consulta utilizado GROUP BY para agrupar las emergencias por tipo

4.2.2. Resultado: Al realizar la ejecución de la consulta SQL, se obtuvieron exitosamente los registros agrupados por tipo de emergencia, mostrando la cantidad de emergencias que han ocurrido dentro del “Instituto Católico Karol Wojtyla”, según su tipo de emergencia.

```

282
283 SELECT
284     i.nombre AS institucion,
285     te.tipo,
286     COUNT(*) AS cant_emergencias
287 FROM public.alertas a
288 LEFT JOIN public.tipos_emergencia te ON a.id_tipo = te.id_tipo
289 INNER JOIN public.ubicaciones u ON a.id_ubicacion = u.id_ubicacion
290 INNER JOIN public.instituciones i ON u.id_institucion = i.id_institucion
291 WHERE i.nombre = 'Instituto Católico Karol Wojtyla'
292 GROUP BY i.nombre, te.tipo;

```

	institucion character varying (125)	tipo character varying (100)	cant_emergencias bigint
1	Instituto Católico Karol Wojt...	Incendio	3
2	Instituto Católico Karol Wojt...	Medica	1
3	Instituto Católico Karol Wojt...	Seguridad	1

Figura 8 Resultado obtenido de la consulta de agregación utilizando GROUP BY.

4.3. Consulta 3: ¿Cuántas emergencias hay en la institución educativa divididas por su estado?

4.3.1. Explicación: Se utilizó LEFT JOIN e INNER JOIN para relacionar los registros de la tabla “alertas” con las tablas “ubicaciones” e “instituciones”. Acompañado de LEFT JOIN e INNER JOIN para relacionar los registros de la tabla “alertas” con las tablas “ubicaciones” e “instituciones”,

permitiendo identificar a qué institución pertenece cada emergencia, permitiendo identificar a qué institución pertenece cada emergencia.

```
295
296 SELECT
297     i.nombre AS institucion,
298     a.estado,
299     COUNT(*) AS cant_emergencias
300 FROM public.alertas a
301 LEFT JOIN public.ubicaciones u ON a.id_ubicacion = u.id_ubicacion
302 INNER JOIN public.instituciones i ON u.id_institucion = i.id_institucion
303 WHERE i.nombre = 'Instituto Católico Karol Wojtyla'
304 GROUP BY i.nombre, a.estado;
305
```

Figura 9 Consulta con GROUP BY para agrupar registros por estado e institución.

4.3.2. Resultado: Al realizar la ejecución de la consulta SQL, se obtuvieron exitosamente 3 registros agrupados por estado, mostrando la cantidad de emergencias existentes en el “Instituto Católico Karol Wojtyla” agrupadas por su estado, como activa, atendida, en proceso y falsa.

```
296 SELECT
297     i.nombre AS institucion,
298     a.estado,
299     COUNT(*) AS total_registradas
300 FROM public.alertas a
301 LEFT JOIN public.ubicaciones u ON a.id_ubicacion = u.id_ubicacion
302 INNER JOIN public.instituciones i ON u.id_institucion = i.id_institucion
303 WHERE i.nombre = 'Instituto Católico Karol Wojtyla'
304 GROUP BY i.nombre, a.estado;
305
```

Data Output Messages Notifications			
Showing rows: 1 to 3			
	institucion character varying (125)	estado character varying (50)	total_registradas bigint
1	Instituto Católico Karol Wojt...	ACTIVA	2
2	Instituto Católico Karol Wojt...	ATENDIDA	2
3	Instituto Católico Karol Wojt...	EN_PROCESO	1

Figura 10 Captura del resultado obtenido de la consulta con GROUP BY.

4.4. Consulta 4: ¿Cuáles son las ubicaciones dentro de la institución que han registrado más de 1 emergencia?

4.4.1. Explicación: Se utilizó COUNT y GROUP BY, para contar y agrupar los registros de emergencias según cada ubicación dentro de la institución educativa. Acompañado con un INNER JOIN para relacionar los registros de la tabla “alertas” con las tablas “ubicaciones” e “instituciones”, permitiendo identificar las zonas donde se han registrado las emergencias. Y para mostrar únicamente las ubicaciones que se han registrado más de una emergencia, se utilizó HAVING COUNT (*) > 1 para cuando realice el conteo solo muestre las emergencias mayores a 1.

```
307
308  SELECT
309      u.nombre AS zona,
310      COUNT(a.id_alerta) AS total_emergencias
311  FROM public.alertas a
312  INNER JOIN public.ubicaciones u ON a.id_ubicacion = u.id_ubicacion
313  INNER JOIN public.instituciones i ON u.id_institucion = i.id_institucion
314  WHERE i.nombre = 'Instituto Católico Karol Wojtyła'
315  GROUP BY u.nombre
316  HAVING COUNT(a.id_alerta) > 1
317  ORDER BY total_emergencias DESC;
```

Figura 11 Consulta SQL con GROUP BY, junto con la condición HAVING

4.4.1. Resultado: Con esto se obtuvieron exitosamente las ubicaciones que han registrado más de una emergencia, mostrando la cantidad total de emergencias existentes en cada ubicación de la institución educativa y ordenándolas de más reciente a la más antigua.

307	
308	SELECT
309	u.nombre AS zona,
310	COUNT(a.id_alerta) AS total_emergencias
311	FROM public.alertas a
312	INNER JOIN public.ubicaciones u ON a.id_ubicacion = u.id_ubicacion
313	INNER JOIN public.instituciones i ON u.id_institucion = i.id_institucion
314	WHERE i.nombre = 'Instituto Católico Karol Wojtyła'
315	GROUP BY u.nombre
316	HAVING COUNT(a.id_alerta) > 1
317	ORDER BY total_emergencias DESC ;
318	

Data Output	Messages	Notifications
-------------	----------	---------------

+ SQL	Showing rows: 1 to 2	Page No: 1												
<table> <tr> <th></th> <th>zona</th> <th>total_emergencias</th> </tr> <tr> <td></td> <td>character varying (180)</td> <td>bigint</td> </tr> <tr> <td>1</td> <td>Edificio A</td> <td>2</td> </tr> <tr> <td>2</td> <td>Edificio B</td> <td>2</td> </tr> </table>		zona	total_emergencias		character varying (180)	bigint	1	Edificio A	2	2	Edificio B	2		
	zona	total_emergencias												
	character varying (180)	bigint												
1	Edificio A	2												
2	Edificio B	2												

Figura 12 Resultado obtenido de la consulta realizada con GROUP BY y HAVING.

5. VALIDACIÓN DE LOS DATOS

5.1. Inconsistencia 1: Incoherencia entre el comando recibido y el estado de la alerta.

El problema identificado, es una inconsistencia entre el estado de una la emergencia con el comando recibido. En el valor de varios registros almacenado en columna comando_recibido no coincide con el estado registrado en la columna “estado”. Por ejemplo, algunos comandos indican que la emergencia se encuentra “Activa”, mientras que el estado de la alerta aparece como “EN_PROCESO”, “ATENDIDA” o “FALSA”. Esto genera una inconsistencia en la información almacenada, ya que ambos datos deberían mantener una relación de acuerdo con el estado actual de la emergencia.

Data Output Messages Notifications			
Showing rows: 1 to 5			
	id_alerta [PK] integer	comando_recibido character varying (50)	estado character varying (50)
1	48	Incendio,Edif_A,Activa	ACTIVA
2	49	Medica,Edif_B,Activa	ACTIVA
3	51	Incendio,Edif_A,Activa	ATENDIDA
4	53	Incendio,Edif_B,Activa	ATENDIDA
5	50	Seguridad,Edif_C,Activa	EN_PROCESO

Figura 13 Inconsistencia 1 de estados contradictorios entre estado y comando.

5.1.1. Corrección aplicada: Se actualizaron los comandos recibidos de las alertas que presentaban información incorrecta utilizando UPDATE, modificando los valores de los registros 50, 51 y 53 para que coincidieran con el estado correspondiente de cada emergencia.

```
SELECT id_alerta, comando_recibido, estado
FROM public.alertas;

UPDATE public.alertas
SET comando_recibido = 'Seguridad,Edif_C,En_proceso'
WHERE id_alerta = 50;

UPDATE public.alertas
SET comando_recibido = 'Incendio,Edif_A,Atendida'
WHERE id_alerta = 51;

UPDATE public.alertas
SET comando_recibido = 'Incendio,Edif_B,Atendida'
WHERE id_alerta = 53;
```

Figura 14 Solución aplicada para inconsistencia 1.

5.1.2. Comprobación: Se realizó nuevamente una consulta SELECT en las columnas comando_recibido y estado para verificar que las modificaciones realizadas con UPDATE se hayan almacenado correctamente y que la información almacenada sea ahora sea coherente con los demás datos registrados de cada emergencia.

Data Output Messages Notifications			
SQL			
	Id_alerta [PK] integer	comando_recibido character varying (50)	estado character varying (50)
1	48	Incendio,Edif_A,Activa	ACTIVA
2	49	Medica,Edif_B,Activa	ACTIVA
3	51	Incendio,Edif_A,Atendida	ATENDIDA
4	50	Seguridad,Edif_C,En_proceso	EN_PROCESO
5	53	Incendio,Edif_B,Atendida	ATENDIDA

Figura 15 Captura de comprobación de datos actualizados correctamente

5.2. Inconsistencia 2: Registros duplicados en la descripción e información de las emergencias

Se identificaron registros relacionados específicamente de tipo incendio, que almacenan la misma información en sus campos de descripción, lo que puede indicar que la información fue duplicada o registrada incorrectamente, esto genera una inconsistencia en los registros de las emergencias. Lo que puede generar confusión al momento de consultar las emergencias y dificulta diferenciar correctamente lo ocurrido en cada ubicación.

Data Output Messages Notifications						
SQL						
	Id_alerta integer	emergencia character varying (100)	ubicacion character varying (180)	usuario_atendio character varying (100)	descripcion character varying (150)	dia_detectad date
1	53	Incendio	Edificio B	Rolando Velasco	Humo detectado cerca del laboratorio principal en Edificio B	2026-08-28
2	51	Incendio	Edificio A	Ana Lucía Gómez	Alarma de incendio activada en pasillo del Edificio A	2026-08-30
3	48	Incendio	Edificio A	Rolando Velasco	Humo detectado cerca del laboratorio principal en Edificio A	2026-09-09

Figura 16 Inconsistencia 2 duplicidad de datos.

5.2.1. Corrección aplicada: Para darle una solución a la inconsistencia identificada en los campos de descripción en las emergencias de tipo incendio con id (53, 51), se realizó una

modificación utilizando UPDATE en la alerta id_alerta = 53, reemplazando la información almacenada en su campo descripción, repetida por una descripción más específica de la emergencia registrada en el Edificio B.

```
360
361 UPDATE public.alertas
362 SET descripcion = 'Fuga de gas detectada en el área de cocina del Edificio B'
363 WHERE id_alerta = 53;
364
```

Data Output Messages Notifications

UPDATE 1

Query returned successfully in 187 msec.

Figura 17 Corrección aplicada para la inconsistencia número 2

5.2.2. Comprobación: Con el propósito de verificar que la información se haya actualizado correctamente en la base de datos, se procedió a realizar una consulta de datos mediante SELECT, para consultar las dos emergencias, que contenían información duplicada o similar en su descripción, con esto se pudo comprobar que ahora ambos registros almacenan información más completa y de forma correcta sin duplicidad de datos:

```
361 -- Comprobación
362
363 SELECT
364     alertas.id_alerta,
365     tipos_emergencia.tipo AS emergencia,
366     ubicaciones.nombre AS ubicacion,
367     alertas.descripcion AS descripcion
368 FROM public.alertas
```

Data Output Messages Notifications

Showing rows: 1 to 3 Page No: 1

	id_alerta integer	emergencia character varying (100)	ubicacion character varying (180)	descripcion character varying (150)
1	53	Incendio	Edificio B	Fuga de gas detectada en el área de cocina del Edificio B
2	51	Incendio	Edificio A	Alarma de incendio activada en pasillo del Edificio A
3	48	Incendio	Edificio A	Humo detectado cerca del laboratorio principal en Edificio A

Figura 18 Comprobación de la solución aplicada a la inconsistencia 2.

5.3. Inconsistencia 3: Falta de restricción de valores en el campo “estado” en la tabla alertas.

Se identificó un problema relacionado con la restricción de valores que se almacenan en la columna “estado”. Esto permite que se puedan ingresar datos incorrectos o escritos de diferentes formas, como “activas”, “Atendida” o “FALSO”, generando inconsistencias en la información almacenada, para evitar posibles errores de datos incorrectos al registrar el estado de cada emergencia y solo acepte datos válidos por el sistema.

```
62 CREATE TABLE IF NOT EXISTS public.alertas(  
63     id_alerta INTEGER UNIQUE GENERATED ALWAYS AS IDENTITY,  
64     id_usuario INTEGER,  
65     id_tipo INTEGER NOT NULL,  
66     id_ubicacion INTEGER NOT NULL,  
67     comando_recibido VARCHAR(50) NOT NULL,  
68     descripcion VARCHAR(150) NOT NULL,  
69     estado VARCHAR(50) NOT NULL DEFAULT 'SEGURO',  
70     hora_emerg TIME NOT NULL DEFAULT (CURRENT_TIME),  
71     fecha DATE NOT NULL DEFAULT (CURRENT_DATE),  
72  
73     CONSTRAINT alertas_pkey PRIMARY KEY(id_alerta),  
74  
75     CONSTRAINT alertas_usuario_fkey  
76     FOREIGN KEY(id_usuario) REFERENCES usuarios(id_usuario),  
77  
78     CONSTRAINT alertas_tipos_fkey  
79     FOREIGN KEY(id_tipo) REFERENCES tipos_emergencia(id_tipo),  
80  
81     CONSTRAINT alertas_ubicaciones_fkey  
82     FOREIGN KEY(id_ubicacion) REFERENCES ubicaciones(id_ubicacion)  
83  
84 );  
85  
86
```

Figura 19 Inconsistencia 3 falta de restricción de datos en columna "estado".

5.3.1. Corrección aplicada: Se agregó una restricción CHECK a la tabla alertas, estableciendo los valores permitidos para la columna estado: ACTIVA, EN_PROCESO, ATENDIDA, FALSA y SEGURO. De esta manera, la base de datos solo permitirá registrar estados que se encuentren dentro de los valores establecidos.

```

377
378 ALTER TABLE public.alertas
379 ADD CONSTRAINT check_estado_valido
380 CHECK (estado IN ('ACTIVA', 'EN_PROCESO', 'ATENDIDA', 'FALSA', 'SEGURO'));
381

```

Data Output Messages Notifications

ALTER TABLE

Query returned successfully in 172 msec.

Figura 20 Corrección aplicada para la inconsistencia 3

5.3.2. Comprobación: Finalmente se para verificar que la creación de la restricción de valores permitidos en la columna “estado” de cada emergencia se procedió a realizar una consulta de las reglas del sistema, con esto se pudo evidenciar que la regla de restricción se creó correctamente.

```

383
384 -- Comprobación:
385
386 SELECT constraint_name, check_clause
387 FROM information_schema.check_constraints
388 WHERE constraint_name = 'check_estado_valido';
389

```

Data Output Messages Notifications

Showing rows: 1 to 1 Page No: 1 of 1

	constraint_name	check_clause
1	check_estado_vali...	((estado)::text = ANY ((ARRAY[ACTIVA::character varying, EN_PROCESO::character varying, ATENDIDA::character varying, FALSA::character varying, SEGURO::character var

6. CONSOLIDACIÓN DEL TRABAJO DEL EQUIPO

Durante el desarrollo de las actividades de trabajo de esta semana, relacionadas con las consultas avanzadas utilizando INNER JOIN, consultas de resumen mediante funciones de agregación y métodos de agrupación, para asegurar que la aportación realizada para el proyecto funcione correctamente, e identificar errores o inconsistencias en la sintaxis SQL utilizada en cada consulta modificación realizada, para darles una solución o mejora para, asegurar que mi aporte al proyecto no producto errores o inconsistencias.

Asimismo, se realizó una revisión y comprobación del correcto funcionamiento de los datos, consultas y relaciones de la base de datos al integrar y verificar la aportación de cada integrante. El objetivo de esta revisión fue identificar posibles errores e inconsistencias durante la integración del trabajo de cada integrante al proyecto y asegurar el correcto funcionamiento de la base de datos.

Nº	Comprobación	Resultado obtenido
1	Consultas INNER JOIN realizadas por cada integrante.	Integración exitosa
2	Consultas resumen utilizando funciones de agregación como (count, sum, min, max, avg y funciones de agrupación de datos con group by.	Integración exitosa
3	Corrección de registros inconsistentes encontrados en el proyecto.	Integración exitosa
4	Relaciones correctas entre las distintas tablas del proyecto.	Integración exitosa
6	Integración de los cambios y trabajo realizado al repositorio del proyecto.	Integración exitosa

7. ACTUALIZACIÓN EN GITHUB

7.1. Branch (rama personal utilizada)

La rama de trabajo utilizada para desarrollar cada una de las prácticas de consultas avanzadas de esta semana y trabajar de manera colaborativa con los demás integrantes en el repositorio del proyecto corresponde a la rama denominada “Rolando_SafeSchool_DB”. Esta es la rama en la cual se estuvo trabajando y publicando cada uno de los cambios, correcciones y aportaciones realizadas al proyecto.

Al finalizar todos los cambios y correcciones correspondientes, serán incorporados a la rama “main”, incluyendo las modificaciones, correcciones y aportaciones correspondientes al trabajo semanal realizado durante esta semana.

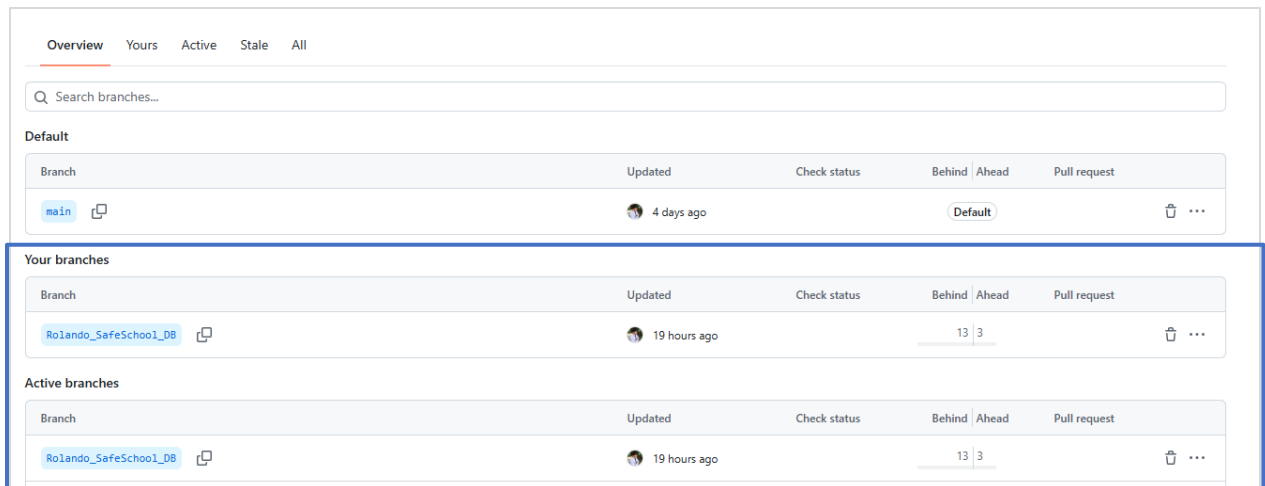


Figura 21 Captura de la rama utilizada durante las actividades.

7.2. Commits

7.2.1. Evidencia de commits: Para evidenciar los commits y las aportaciones realizadas al proyecto, se presenta en la siguiente captura los commits creados para actualizar y publicar nuevos cambios, así como las actualizaciones de las sintaxis SQL utilizadas para la consulta de datos y la modificación y eliminación de registros en la tabla alertas.

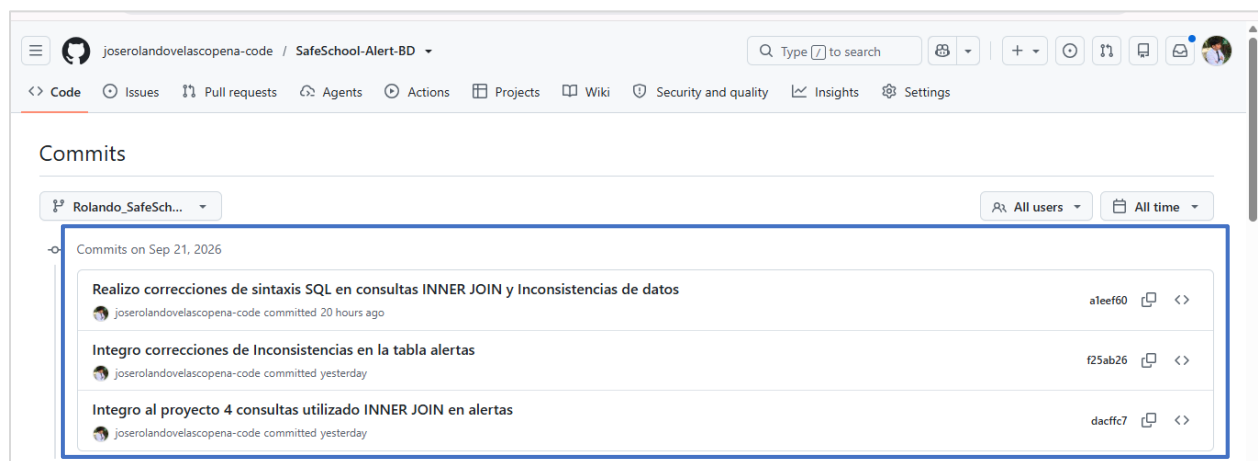


Figura 22 Evidencia de commits realizados en el repositorio.

7.3. Archivos modificados

En esta sección se presentan los archivos modificados en cada commit realizado para publicar nuevos cambios y correcciones en los archivos “03_consultas_alertas” y “02_datos_prueba_alertas”, que son los dos principales archivos con los que se ha trabajado en

las consultas utilizando INNER JOIN, consultas de resumen, modificaciones y correcciones de datos inconsistentes. Para ello, a continuación, se presentan los archivos modificados en los que se ha trabajado cada una de las consultas y modificaciones presentadas en este informe.

7.3.1. Archivo: 03_consultas_alertas

```

224 +
225 + -- SEMANA 3: CONSULTAS
226 +
227 + -- CLASE 1 SEMANA 3: Consultas relacionadas entre tablas
228 +
229 + -- CONSULTA INNER JOIN 1: Mostrar información más completa de emergencias, mostrando usuario
230 + -- que la atendio, ubicación y tipo.
231 +
232 + SELECT
233 + alertas.id_alerta,
234 + tipos_emergencia.tipo AS emergencia,
235 + usuarios.nombre AS usuario_atendio,
236 + alertas.descripcion AS descripcion,
237 + ubicaciones.nombre AS ubicacion,
238 + alertas.fecha AS dia_detectada,
239 + alertas.hora_emerg AS hora,
240 + alertas.estado
241 + FROM public.alertas
242 + LEFT JOIN public.tipos_emergencia ON alertas.id_tipo = tipos_emergencia.id_tipo
243 + INNER JOIN public.usuarios ON alertas.id_usuario = usuarios.id_usuario
244 + INNER JOIN public.ubicaciones ON alertas.id_ubicacion = ubicaciones.id_ubicacion;
245 +
246 + -- CONSULTA INNER JOIN 2: Mostrar todas las ubicaciones del Edificio A, mostrando
247 + -- el tipo de la emergencia, fecha y hora, de la más reciente a la más antigua.
  
```

Figura 23 Archivo modificado "02_datos_prueba_alertas".

7.3.1. Archivo: 03_consultas_alertas

```

15 15 fecha)
16 16 VALUES
17 17 (1, 1, 1, 'Incendio,Edif_A,Activa', 'Humo detectado cerca del laboratorio principal en Edificio A', 'ACTIVA', '10:30:00', '2026-
09-09'),
18 - (2, 2, 2, 'Medica,Edif_B,Activa', 'Estudiante herido en la cancha central del Edificio B', 'ACTIVA', '4:30:00', '2026-09-01'),
18 + (2, 2, 2, 'Medica,Edif_B,Activa', 'Estudiante herido en la cancha central del Edificio B', 'ACTIVA', '04:30:00', '2026-09-01'),
19 19 (3, 3, 3, 'Seguridad,Edif_C,Activa', 'Persona no autorizada detectada en el acceso al Edificio C', 'EN_PROCESO', '11:45:00',
'2026-09-02'),
20 20 (4, 1, 1, 'Incendio,Edif_A,Activa', 'Alarma de incendio activada en pasillo del Edificio A', 'ATENDIDA', '12:20:00', '2026-08-
30'),
21 - (3, 2, 1, 'Medica,Edif_A,Activa', 'Desmayo de alumno en el primero Sección "A" del Edificio A', 'FALSA', '15:10:00', '2026-08-
28'),
21 + (3, 2, 1, 'Medica,Edif_A,Activa', 'Desmayo de alumno en Primer Año Sección "A" del Edificio A', 'FALSA', '15:10:00', '2026-08-
28'),
22 22 (1, 1, 2, 'Incendio,Edif_B,Activa', 'Humo detectado cerca del laboratorio principal en Edificio B', 'ATENDIDA', '15:10:00',
'2026-08-28'),
23 - (3, 2, 2, 'Medica,Edif_B,Activa', 'Estudiante lesionado en la cancha.', 'FALSA', '5:10:00', '2026-08-24');
23 + (3, 2, 2, 'Medica,Edif_B,Activa', 'Estudiante lesionado en la cancha.', 'FALSA', '05:10:00', '2026-08-24');
24 24
  
```

Figura 24 Archivo modificado "03_consultas_alertas".

8. REGISTRO DE MI APOORTE



BASE DE DATOS PARA SAFESHOOL ALERT:

Datos personales y del proyecto

Proyecto:	SafeSchool Alert
Nombre del estudiante:	José Rolando Velasco Peña
Rol asignado:	Consultas de alertas y emergencias (Tabla “alertas”) y detección de datos inconsistentes en alertas.

Mi aporte durante la Semana 3

Durante esta semana desarrollé consultas SQL utilizando INNER JOIN, para obtener información más completa de cada emergencia, gracias a relaciones entre las diferentes tablas como: “ubicaciones, instituciones, y tipos de alertas”, de este modo se puede obtener datos más exactos de cada emergencia, como el tipo de emergencia, la ubicación donde se originó la emergencia, el usuario que atendió la emergencia etc.

Además, realicé consultas de resumen utilizando funciones de agregación, y realicé una búsqueda y una validación de los datos almacenados en la tabla alertas, con el objetivo de encontrar datos inconsistentes, o estructuras incorrectas de registro de datos.

Consultas desarrolladas:

- **Consulta 1:** Consulta avanzada utilizando INNER JOIN y LEFT JOIN para mostrar información detallada de las emergencias, el usuario que atendió la emergencia, el tipo de emergencia, la ubicación en que se originó la emergencia y la institución correspondiente.

- **Consulta 2:** Una consulta utilizando INNER JOIN, WHERE y ORDER BY para mostrar las emergencias registradas en el Edificio A, mostrando su tipo, descripción, fecha, hora y estado, además ordenadas de la más reciente a la más antigua.
- **Consulta 3:** Una consulta de tipo resumen utilizando COUNT, GROUP BY y HAVING para obtener la cantidad de emergencias según su tipo, estado y ubicación dentro del Instituto Católico Karol Wojtyla.

Problemas o inconsistencias encontradas:

Mediante la revisión en busca de datos, errores y campos incompletos, se identificaron inconsistencias entre los campos registrado en comando_recibido y el estado de las emergencias. Se identificaron registros con información duplicada o similar en el campo de descripción en dos emergencias de tipo incendio, que almacenaban la misma descripción cuando la emergencia se detectó en diferentes ubicaciones y día. Y faltas de restricción de registro de datos para el campo estado de cada emergencia para que acepte solo valores permitidos por el sistema.

¿Cómo los solucioné?

Para solucionar cada una de las diferentes inconsistencias encontradas en la revisión, se realizaron modificaciones mediante la sentencia UPDATE en cada registro con información duplica o errónea, y utilizado ALTER TABLE, para añadir restricciones de datos permitidos por el sistema en el campo “estado” de cada emergencia.

¿Cómo los solucioné?

Lo que aprendí en esta práctica fue a obtener datos de emergencias completos, consultado información en tablas externas utilizando INNER JOIN, y a crear consultas complejas y útiles para mostrar información resumida utilizando funciones de agregación y agrupación de datos mediante COUNT y GROUP BY.

9. CONCLUSIÓN

En conclusión, durante esta actividad se pudo lograr desarrollar consultas avanzadas utilizando INNER JOIN, útiles que le dan respuesta a una necesidad real que la aplicación necesitara saber para gestionar y darle un seguimiento a cada emergencia. Además, pude aprender a utilizar consultas de agregación y agrupación de datos para obtener y resumir información de las emergencias, así como identificar y corregir inconsistencias encontradas en los datos de la tabla alertas. También se pudo mejorar la estructura de la base de datos al implementar restricciones en el registro de datos en la tabla de estado de cada emergencia que el sistema pueda entender.